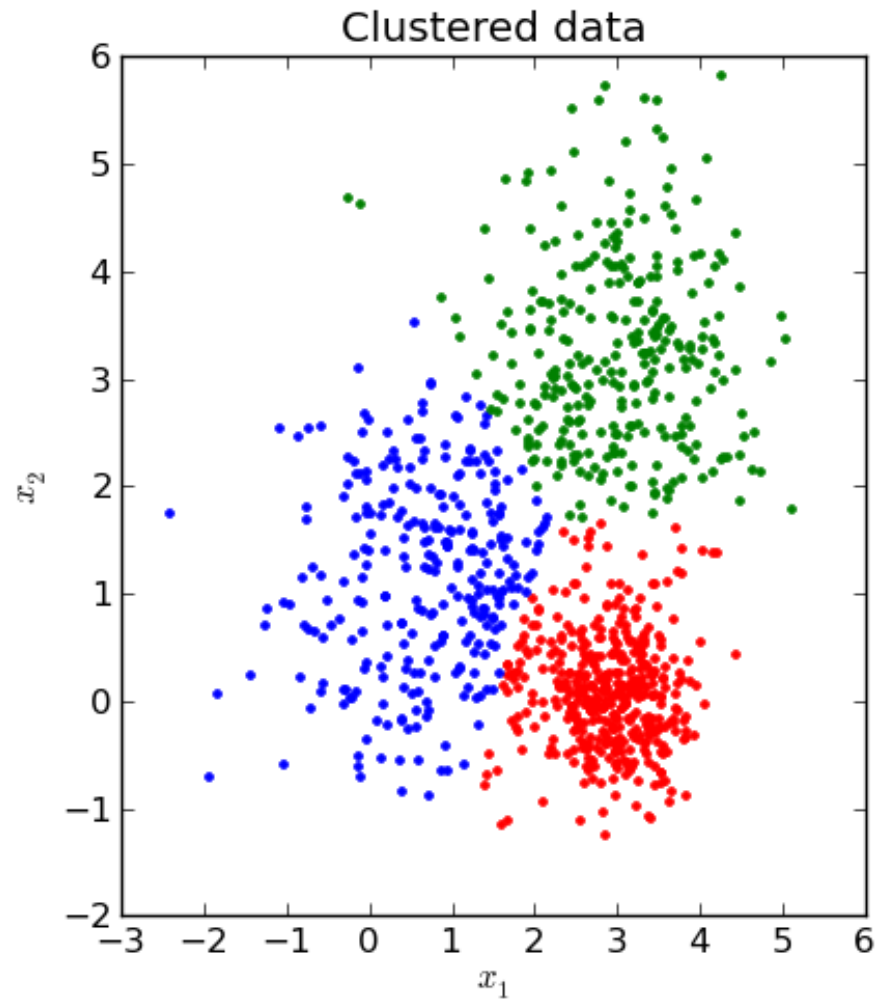
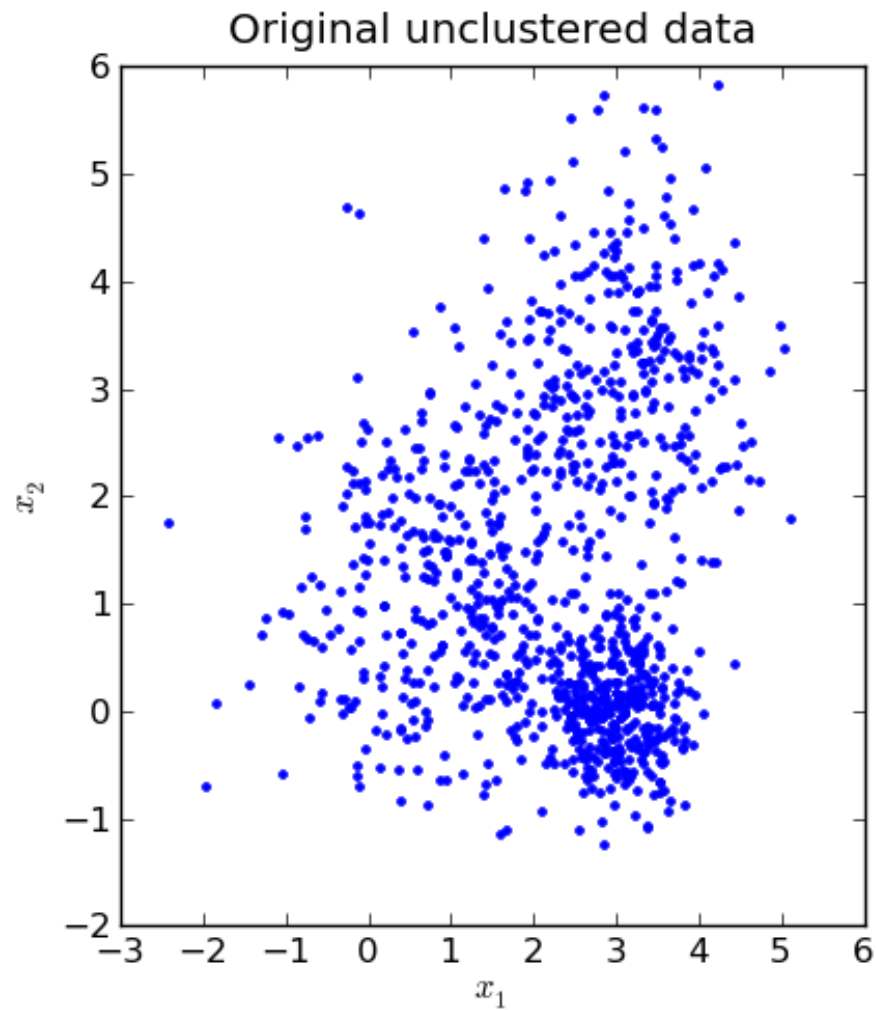


Unsupervised Learning

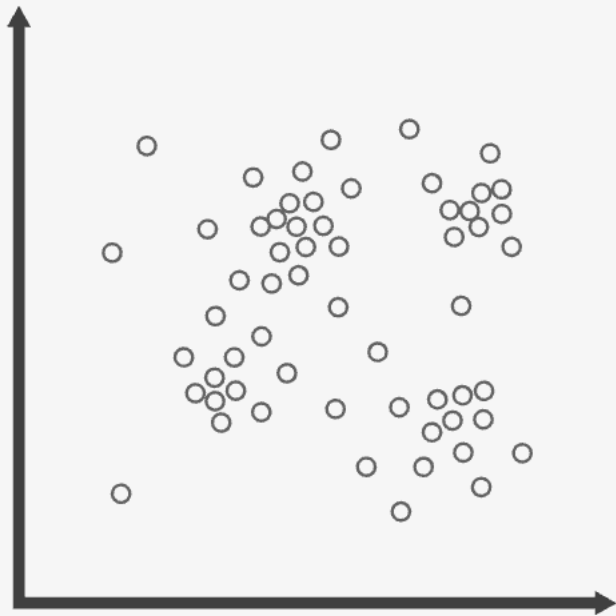
Unsupervised Learning

- Unsupervised learning is a key branch of machine learning that operates on unlabelled and unclassified data to uncover hidden patterns, structures, or relationships.
- Unlike supervised learning, it does not use known outcomes for training.
- The goal is to identify meaningful insights, such as groups of similar objects or correlations among variables.
- Algorithms in this category include Clustering and Association Analysis.
- These techniques help organize information when prior knowledge is unavailable.
- Real-world data often lacks labels — unsupervised learning enables systems to discover the underlying structure.

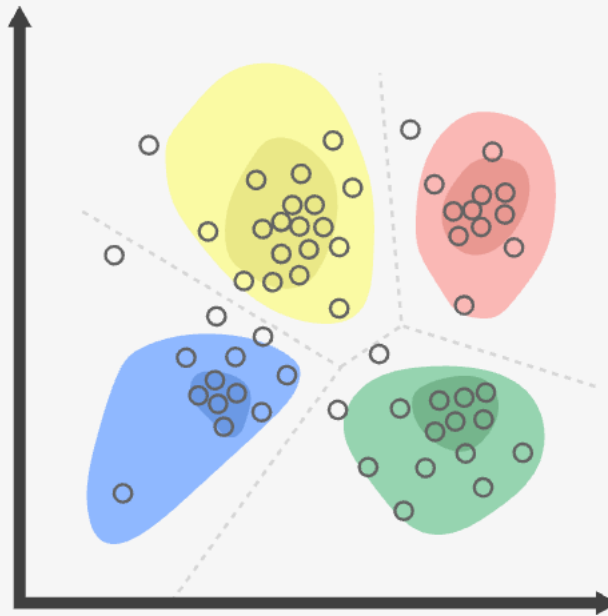
Unsupervised Learning



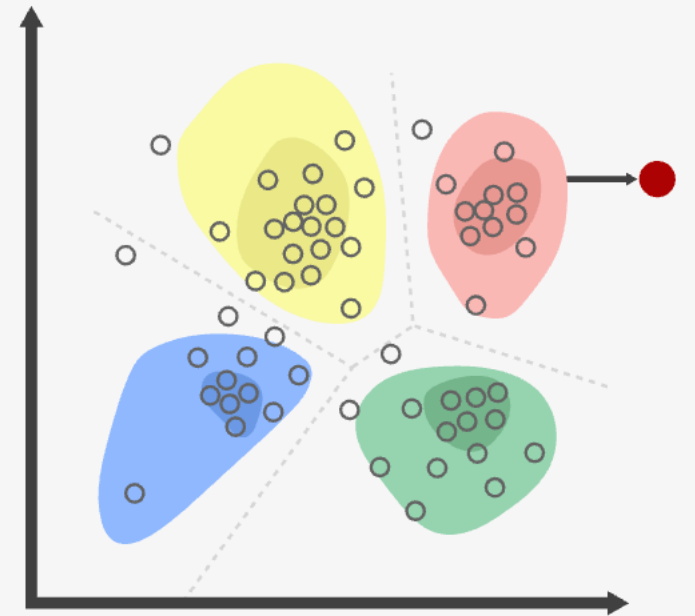
Unsupervised Learning



Training Data



Resulting Model



Applied to New Input

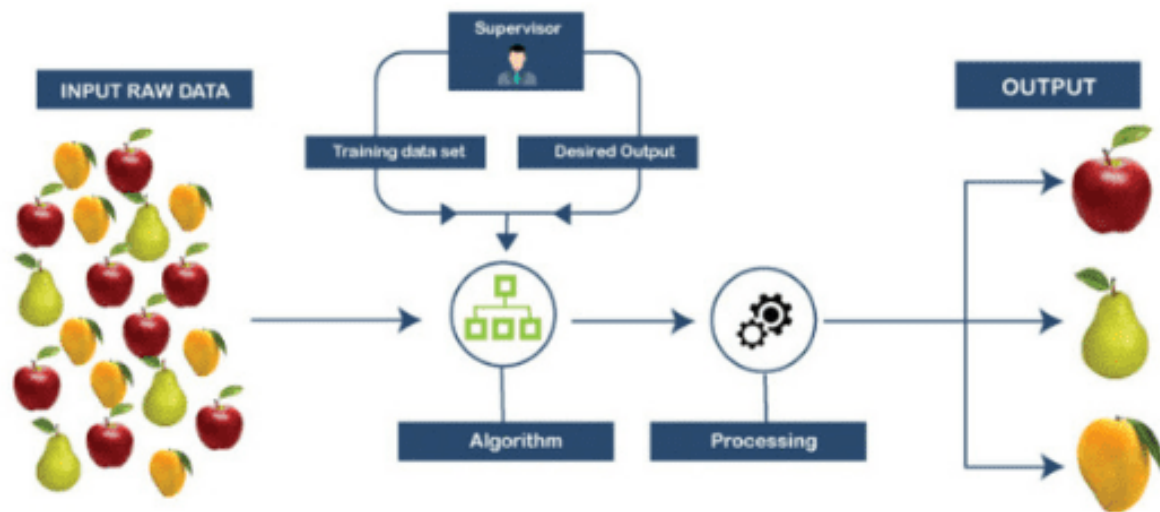
Supervised vs. Unsupervised Learning

- In supervised learning, the algorithm learns a mapping from input features (X) to output labels (Y).
- The purpose is prediction—for example, classification and regression.
- In contrast, unsupervised learning only deals with X (features) and seeks to find patterns or associations without predefined targets.
- Supervised learning estimates a posterior probability $P(Y|X)$, whereas unsupervised learning models joint probability $P(X, Y)$ to understand data structure.
- Example: movie recommendation systems use clustering to group users by preferences.

Supervised vs. Unsupervised Learning

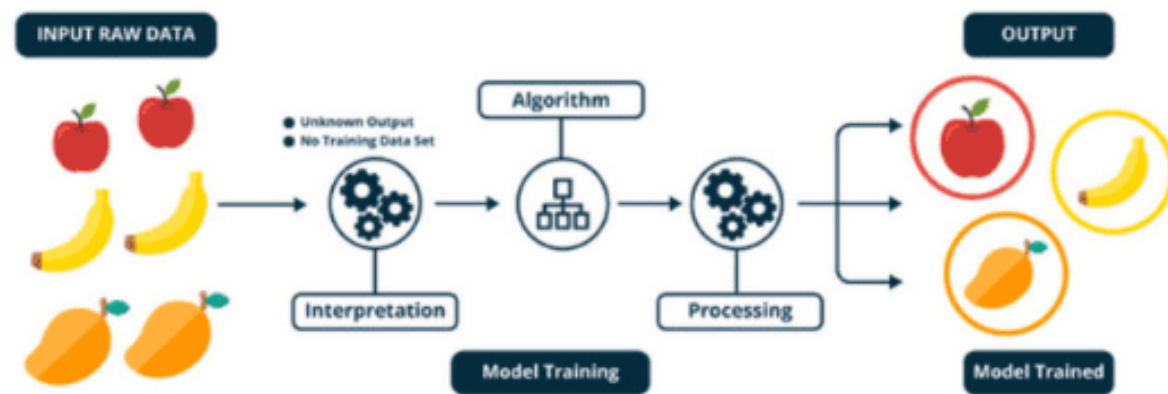
Factor	Supervised Learning	Unsupervised Learning
Data Type	Labeled data: Each input has a corresponding output.	Unlabeled data: No predefined output labels.
Goal	Train models to predict outcomes or classify data.	Identify hidden structures and relationships in data.
Common Tasks	Regression, classification.	Clustering, dimensionality reduction, anomaly detection.
Training Approach	Relies on predefined labels for learning.	Learns from patterns without supervision.
Performance Metrics	Accuracy, F1-score, precision, recall, RMSE, R-squared.	Silhouette score, inertia, or external validation (if labels exist).
Notable Algorithms	Random forests, support vector machines, deep learning.	K-means clustering, hierarchical clustering, autoencoders.
Real-World Uses	Spam detection, medical imaging, stock price prediction.	Market segmentation, fraud detection, recommendation engines.

SUPERVISED LEARNING



VS

UNSUPERVISED LEARNING



Applications of Unsupervised Learning

Unsupervised learning is used in numerous domains:

- Customer segmentation for marketing and targeted advertising.
- Fraud and anomaly detection in banking.
- Image processing tasks such as facial recognition and segmentation.
- Genetics—grouping genes with similar expression patterns.
- Document clustering to organize large text corpora.
- Dimensionality reduction to simplify datasets for modelling.

It is also integrated with Artificial Intelligence systems like chatbots and self-driving cars.

Clustering Concepts

- Clustering identifies groups (clusters) in data where intra-group similarity is high and inter-group difference is high.
- It is unsupervised because no prior labels exist.
- The main types are Partitioning (e.g., K-means, K-medoids), Hierarchical, and Density-based (DBSCAN) methods.
- Each defines a clustering criterion differently.
- For instance, K-means minimizes the Sum of Squared Error (SSE) within clusters.
- Clustering helps reduce data complexity and find structural patterns.

Fields Where Cluster Analysis Is Used Effectively

- **Marketing & Retail:** Businesses use cluster analysis for customer segmentation, dividing their customer base into groups with similar purchasing behaviors or demographics. This enables targeted marketing, personalized offers, and optimized product recommendations. For example, e-commerce platforms cluster shoppers by buying habits to tailor promotions.
- **Healthcare:** Hospitals and researchers use clustering to group patients by disease severity, treatment response, or genetic profiles. This helps in personalizing care plans and identifying subgroups for clinical studies. Epidemiologists also use clustering to detect disease outbreaks in specific regions.
- **Social Media & Digital Platforms:** Platforms cluster users based on engagement levels or content preferences, allowing for personalized feeds and targeted notifications. This increases user engagement and satisfaction.

Fields Where Cluster Analysis Is Used Effectively

- City Planning & Transportation: Urban planners use clustering to allocate resources efficiently, such as identifying neighborhoods needing more schools or improved transit based on demographic clusters.
- Sports Science: Teams cluster athletes by performance metrics to design specialized training and strategies.
- Finance & Banking: Clustering helps in fraud detection by grouping similar transaction patterns and identifying anomalies.
- Other Fields: Real estate (grouping properties), biology (gene expression analysis), and IoT (sensor data grouping) also benefit from cluster analysis.

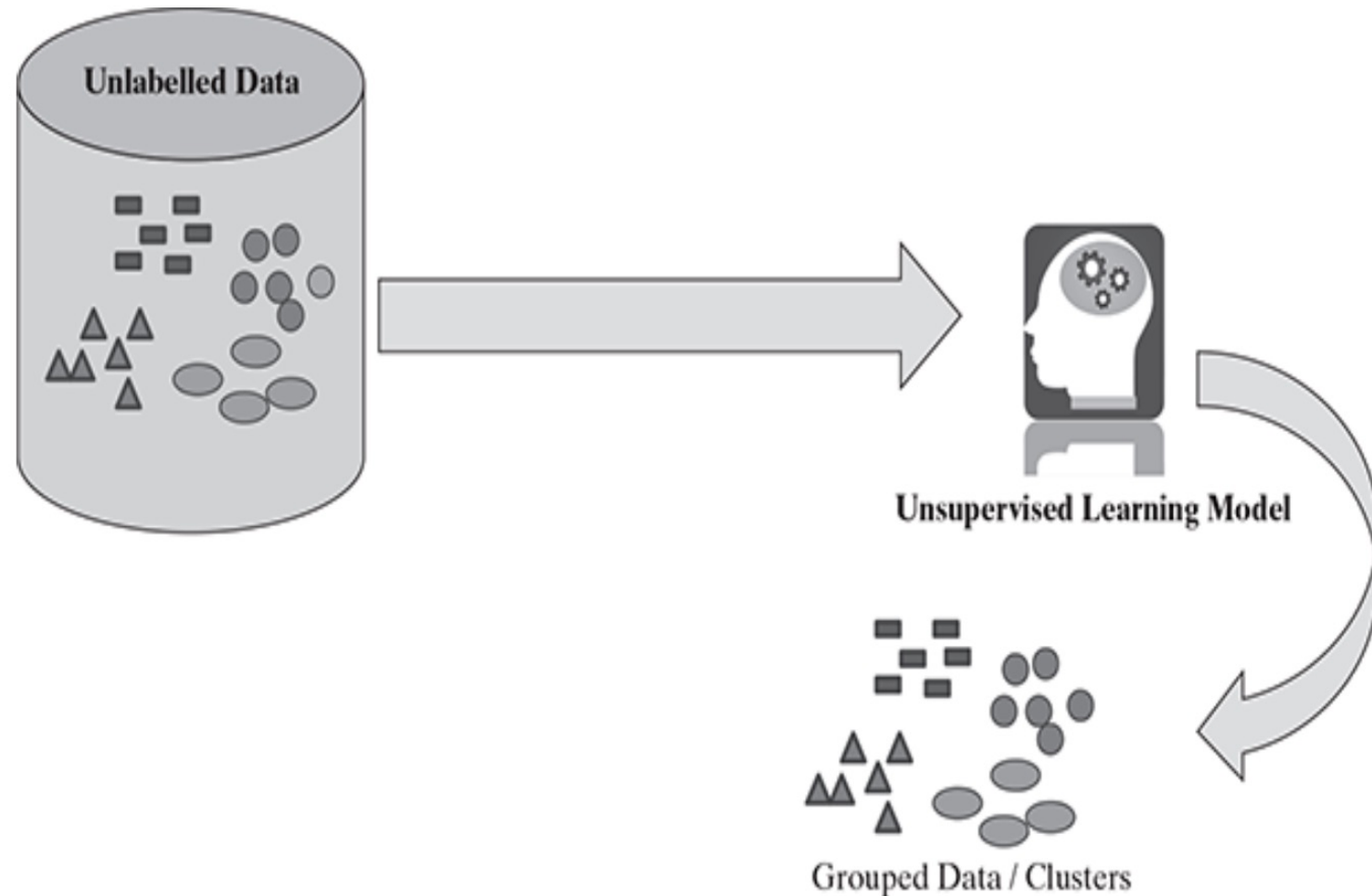
Clustering as a Machine Learning Task

- Clustering is a core unsupervised machine learning task where the goal is to group data points into clusters such that points within the same cluster are more similar to each other than to those in other clusters.
- Unlike supervised learning, clustering does not rely on labelled data; instead, it discovers structure in the data based on feature similarity.

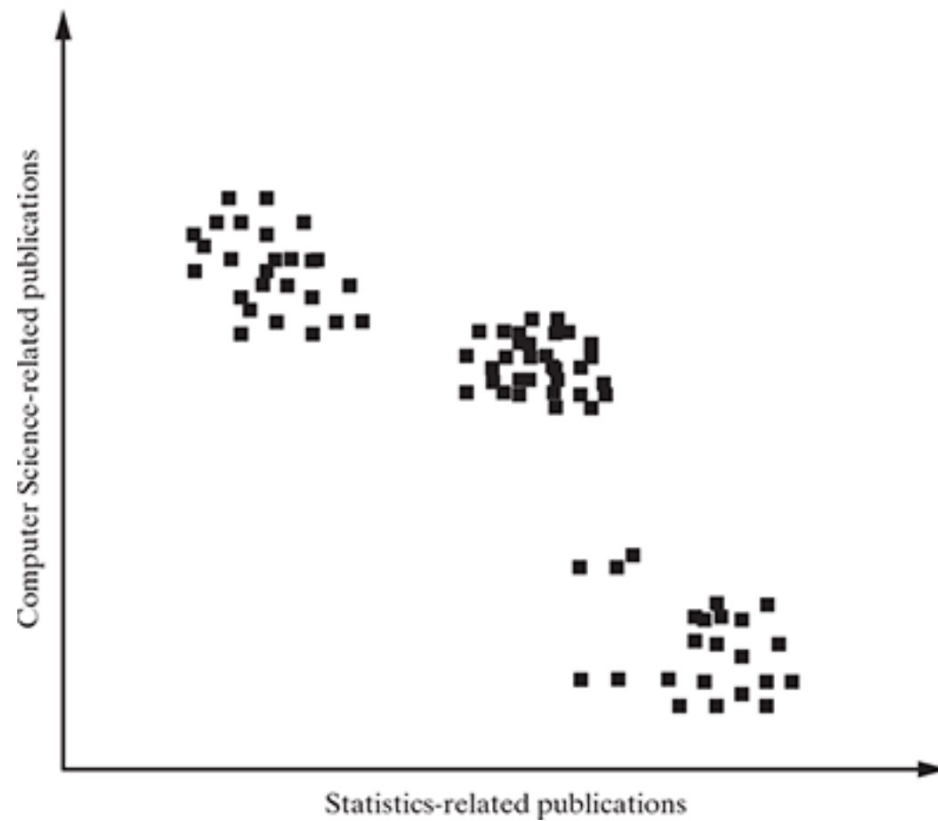
Clustering as a Machine Learning Task

- Input: Only feature data (no labels).
- Output: A set of clusters, each containing similar data points.
- Similarity Measures: Algorithms use distance metrics (like Euclidean or Manhattan distance) to assess similarity.
- Types of Clustering Algorithms:
 - *Partitioning methods* (e.g., K-means, K-medoids): Assign data into a predefined number of clusters.
 - *Hierarchical methods*: Build nested clusters in a tree-like structure.
 - *Density-based methods* (e.g., DBSCAN): Find clusters of arbitrary shape based on data density.
- Applications: Clustering is used for data exploration, anomaly detection, data compression, and as a preprocessing step for other machine learning tasks.

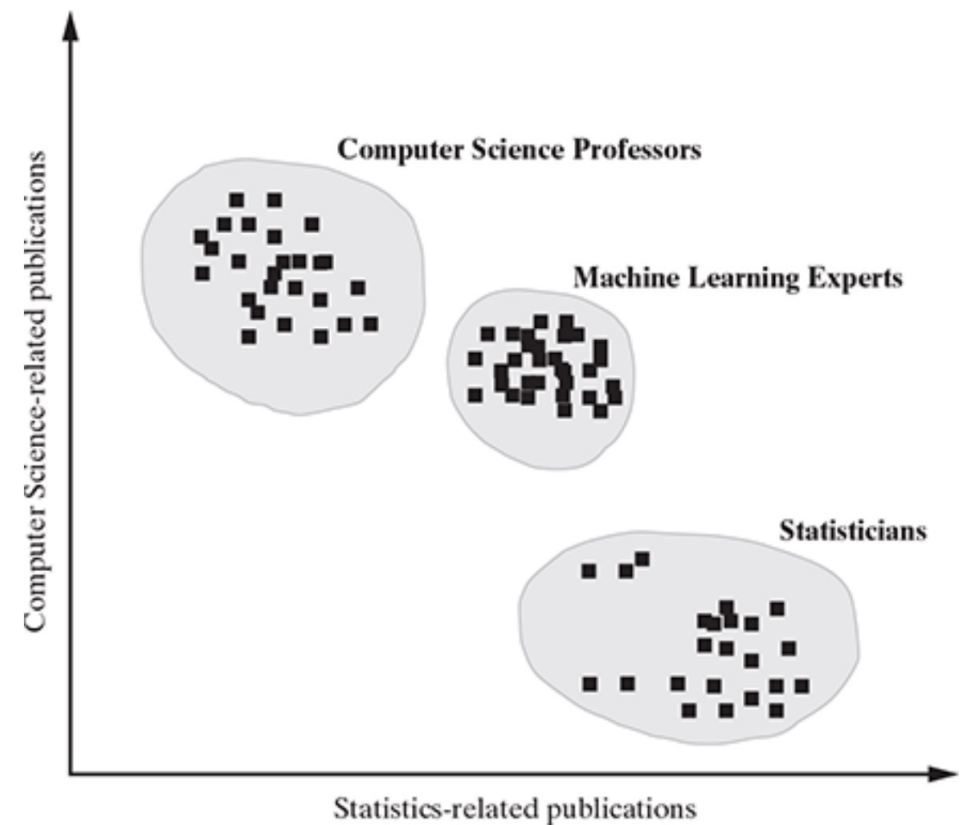
Clustering as a Machine Learning Task



Clustering as a Machine Learning Task



Data set for the conference attendees



Clusters for the conference attendees

Different Types of Clustering Techniques

Clustering techniques in machine learning are diverse, each suited to different data structures and analysis goals. The main types are:

- **Partitioning Methods:** These divide data into a predefined number of clusters. The most common algorithms are K-means (uses centroids as cluster centers) and K-medoids (uses actual data points as centers). These methods are efficient for large datasets but require the number of clusters to be set in advance and are sensitive to outliers.
- **Hierarchical Methods:** These build a tree-like structure (dendrogram) of nested clusters. There are two approaches: *agglomerative* (bottom-up, merging clusters) and *divisive* (top-down, splitting clusters). Hierarchical clustering does not require the number of clusters to be specified beforehand and is useful for visualizing data structure, but can be computationally intensive for large datasets.
- **Density-Based Methods:** These identify clusters as dense regions of data separated by sparser areas. DBSCAN is a popular algorithm that can find clusters of arbitrary shape and handle noise well. It does not require the number of clusters to be set in advance but can be sensitive to parameter choices.

Different Types of Clustering Techniques

Method	Characteristics
Partitioning methods	• Uses mean or medoid (etc.) to represent cluster centre • Adopts distance-based approach to refine clusters • Finds mutually exclusive clusters of spherical or nearly spherical shape • Effective for data sets of small to medium size
Hierarchical methods	• Creates hierarchical or tree-like structure through decomposition or merger • Uses distance between the nearest or furthest points in neighbouring clusters as a guideline for refinement • Erroneous merges or splits cannot be corrected at subsequent levels
Density-based methods	• Useful for identifying arbitrarily shaped clusters • Guiding principle of cluster creation is the identification of dense regions of objects in space which are separated by low-density regions • May filter out outliers

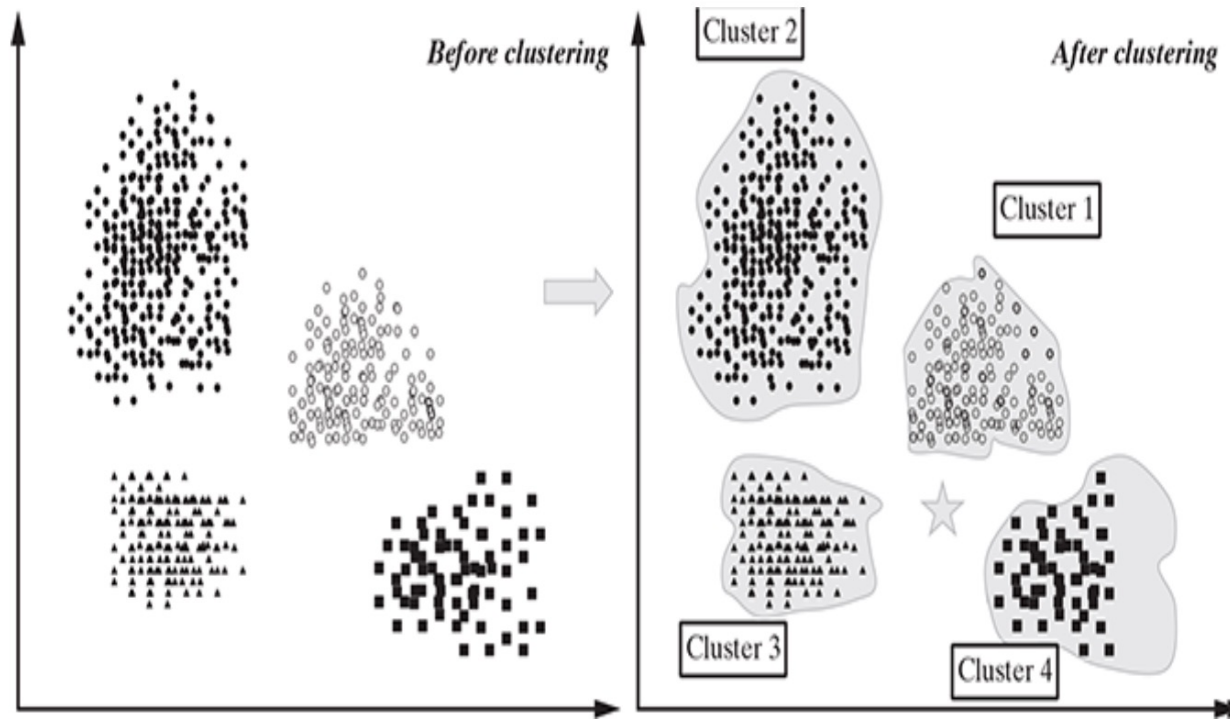
Partitioning Methods in Clustering

- Partitioning methods are a fundamental approach in clustering, where the dataset is divided into a set number of clusters based on similarity.
- The two most widely used partitioning algorithms are K-Means and K-Medoids.
- Each has distinct characteristics, strengths, and limitations, making them suitable for different types of data and analysis needs.

K-Means Clustering

- Concept: K-Means partitions data into k clusters by minimizing the sum of squared distances (SSE) between data points and their cluster centroids. Centroids are the mean position of all points in a cluster and may not correspond to actual data points.
- Algorithm Steps:
 1. Initialize k centroids randomly.
 2. Assign each data point to the nearest centroid.
 3. Recalculate centroids as the mean of assigned points.
 4. Repeat steps 2–3 until centroids stabilize.
- Strengths: Fast, efficient for large datasets, works well with numerical data.
- Limitations: Sensitive to outliers, assumes spherical clusters, requires k to be specified in advance.

K-Means Clustering



K-Means Algorithm Initialization

The process begins by setting the desired number of clusters, K , to 4.

Step 1: Initial Centroid Assignment and Clustering

Four **random points** from the dataset are initially selected to serve as the cluster **centroids**. Following this selection, every data point is then assigned to the cluster of the **nearest centroid**, which forms the initial four clusters.

Step 2: Centroid Update and Reassignment

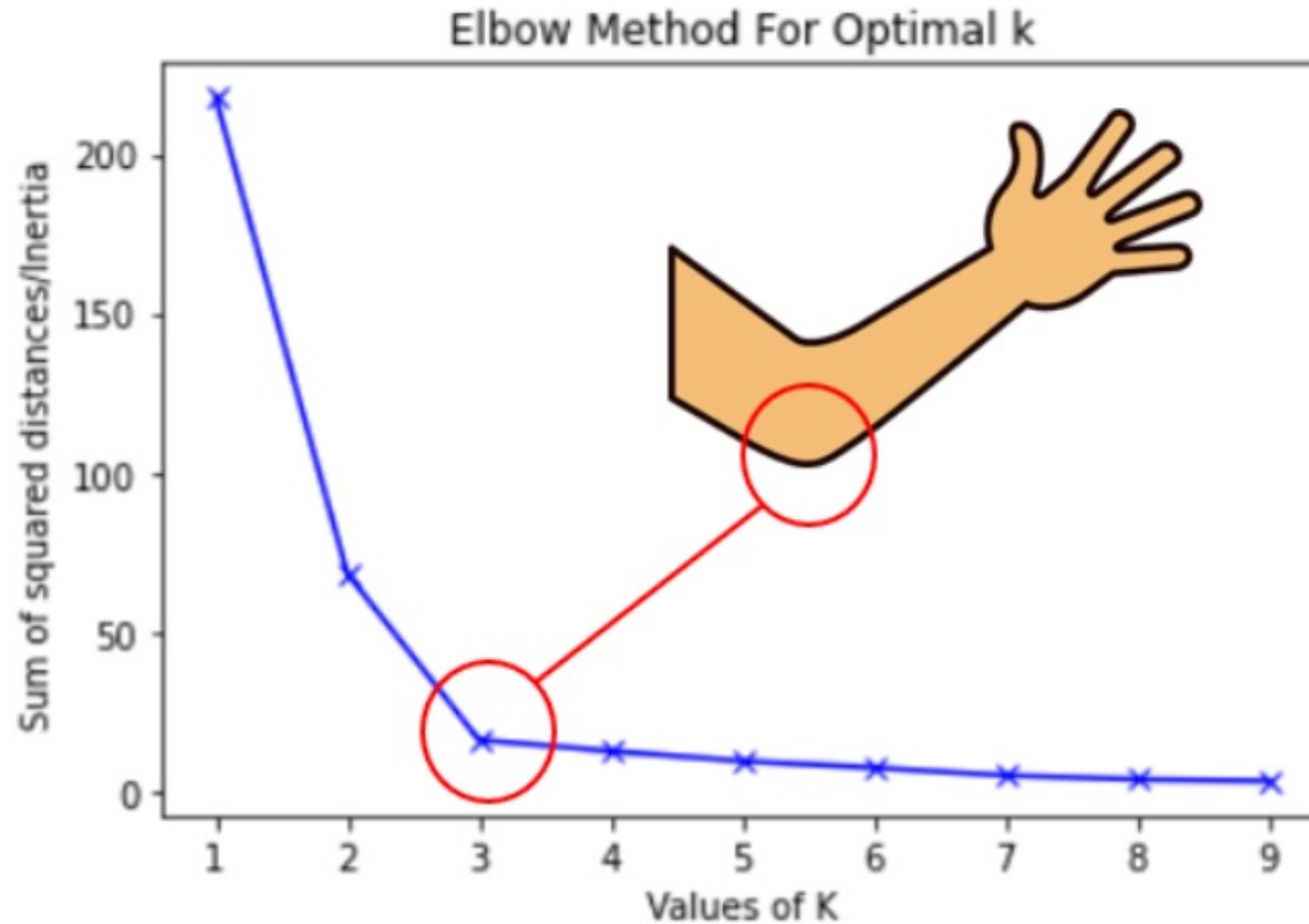
In the second step, the algorithm refines the clusters. The **centroids** are **updated** based on the current cluster assignments, taking into account the distance of the points from their corresponding centroid (typically, the new centroid becomes the mean position of all points in its cluster).

After the centroids are updated, the data points are then **reassigned** to the new nearest centroid, further refining the cluster boundaries. This iterative process of updating centroids and reassigning points continues until the cluster assignments stabilize.

Choosing the Appropriate Number of Clusters

- Domain Knowledge: If you have prior information about the data (e.g., number of departments in a university), use it to set K for meaningful segmentation.
- Business Needs: Sometimes, practical constraints or business requirements dictate the number of clusters (e.g., movie makers clustering by budget and casting, resulting in four logical groups).
- Rule of Thumb: For small datasets, a common heuristic is to set K as the square root of half the number of data points ($K = \sqrt{n/2}$), though this is not reliable for large datasets.
- Statistical Methods: The Elbow Method is widely used to determine the optimal K . It involves plotting the sum of squared errors (SSE) for different values of K and identifying the point where the rate of decrease sharply changes—the "elbow." This point suggests the best trade-off between cluster homogeneity and computational effort.
- Iterative Refinement: Multiple runs with different K values and initial centroids help in finding the most suitable clustering solution

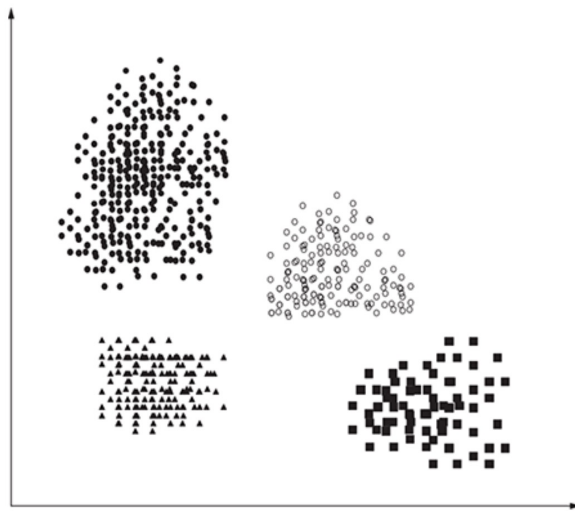
Choosing the Appropriate Number of Clusters



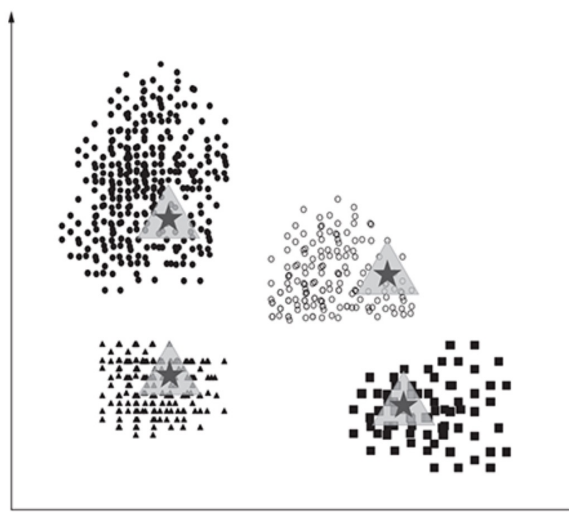
Line plot between K and inertia

Recomputing Cluster Centroids

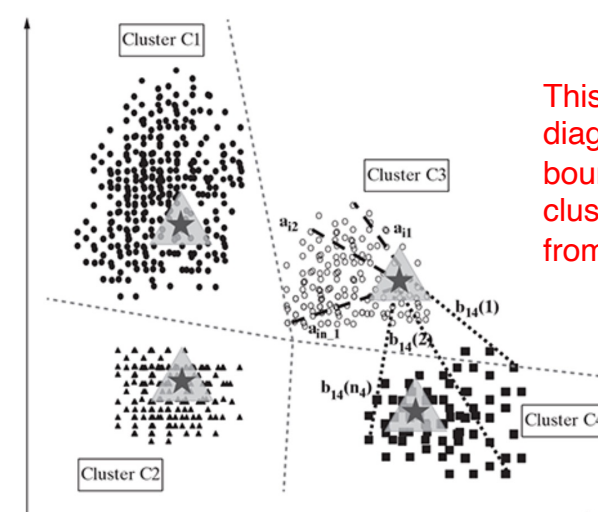
- Recomputing cluster centroids is a crucial step in partitioning clustering algorithms like K-means.
- This process ensures that clusters are optimally positioned to represent the data, improving both homogeneity within clusters and separation between clusters.
- Iterative Refinement: After each assignment of data points to clusters, the centroid of each cluster is recalculated. The centroid is the mean position of all points in the cluster, and this step minimizes the sum of squared errors (SSE) within clusters.
- Distance Calculation: The proximity of each data point to its cluster centroid is measured, typically using **Euclidean distance**. The goal is to minimize the distance between points and their assigned centroid.
- SSE Formula: The quality of clustering is evaluated using the SSE, which sums the squared distances between each point and its cluster centroid across all clusters. Lower SSE indicates better clustering
- Handling Outliers: The mean (centroid) can be distorted by outliers, affecting cluster quality. Alternative methods like K-medoids use actual data points as cluster centers to reduce this effect.
- Voronoi Diagram: After recomputing centroids, cluster boundaries are updated, often visualized using a Voronoi diagram that shows the partitioning of data space.



Clustering of data set

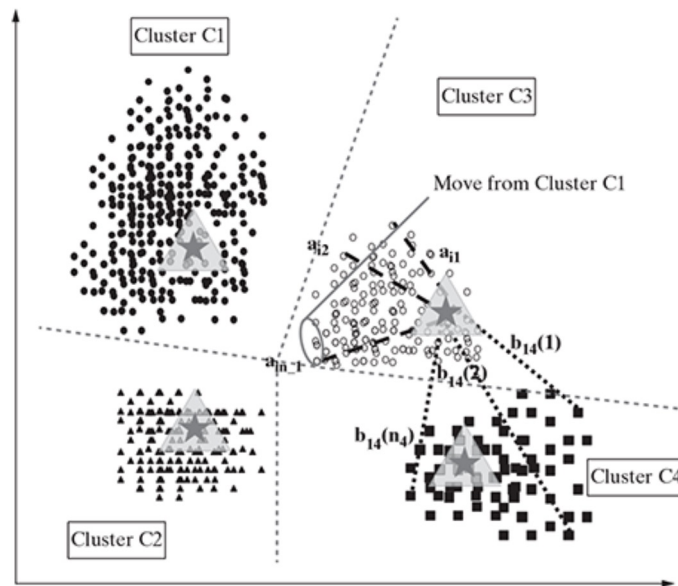


Clustering with initial centroids

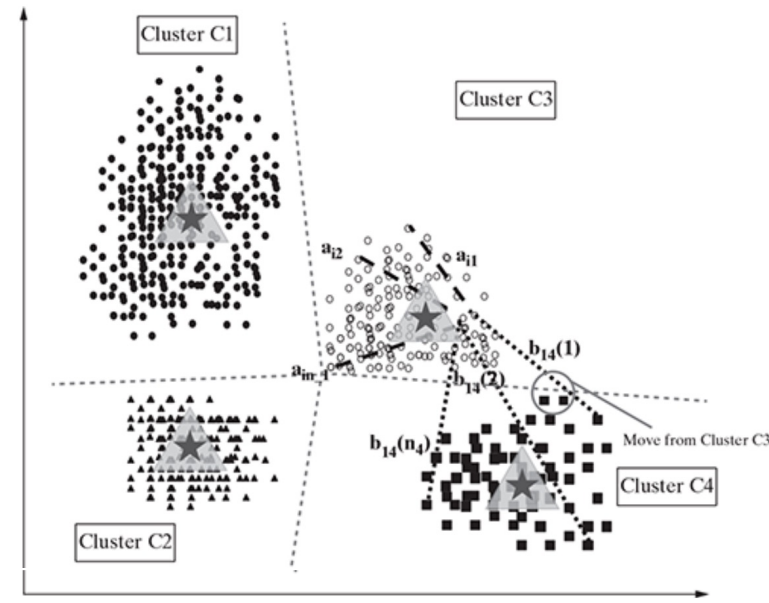


Iteration 1: Four clusters and distance of points from the centroids

This diagram is called Voronoi diagram which creates the boundaries of the clusters. initial four clusters, created by the dashed lines from the vertex of the clusters



Iteration 2: Centroids recomputed and points redistributed among the clusters according to the nearest centroid



Iteration 3: Final cluster arrangement: Centroids recomputed and points redistributed among the clusters according to the nearest centroid

K-Means and K-Medoids

- K-means assigns each of n data points to one of K clusters, then iteratively updates cluster centroids to minimize SSE.
- It's efficient but sensitive to outliers.
- K-medoids, on the other hand, uses representative data points (medoids) rather than means, thus being more robust to noise.
- K-medoids' implementation, Partitioning Around Medoids (PAM), swaps representatives until SSE no longer decreases.

K-Means and K-Medoids

Aspect	K-Means	K-Medoids
Cluster Center	Centroid (mean)	Medoid (actual point)
Sensitivity to Outliers	High	Low
Distance Metric	Usually Euclidean	Any (e.g., Manhattan)
Speed	Fast	Slower
Interpretability	Lower	Higher

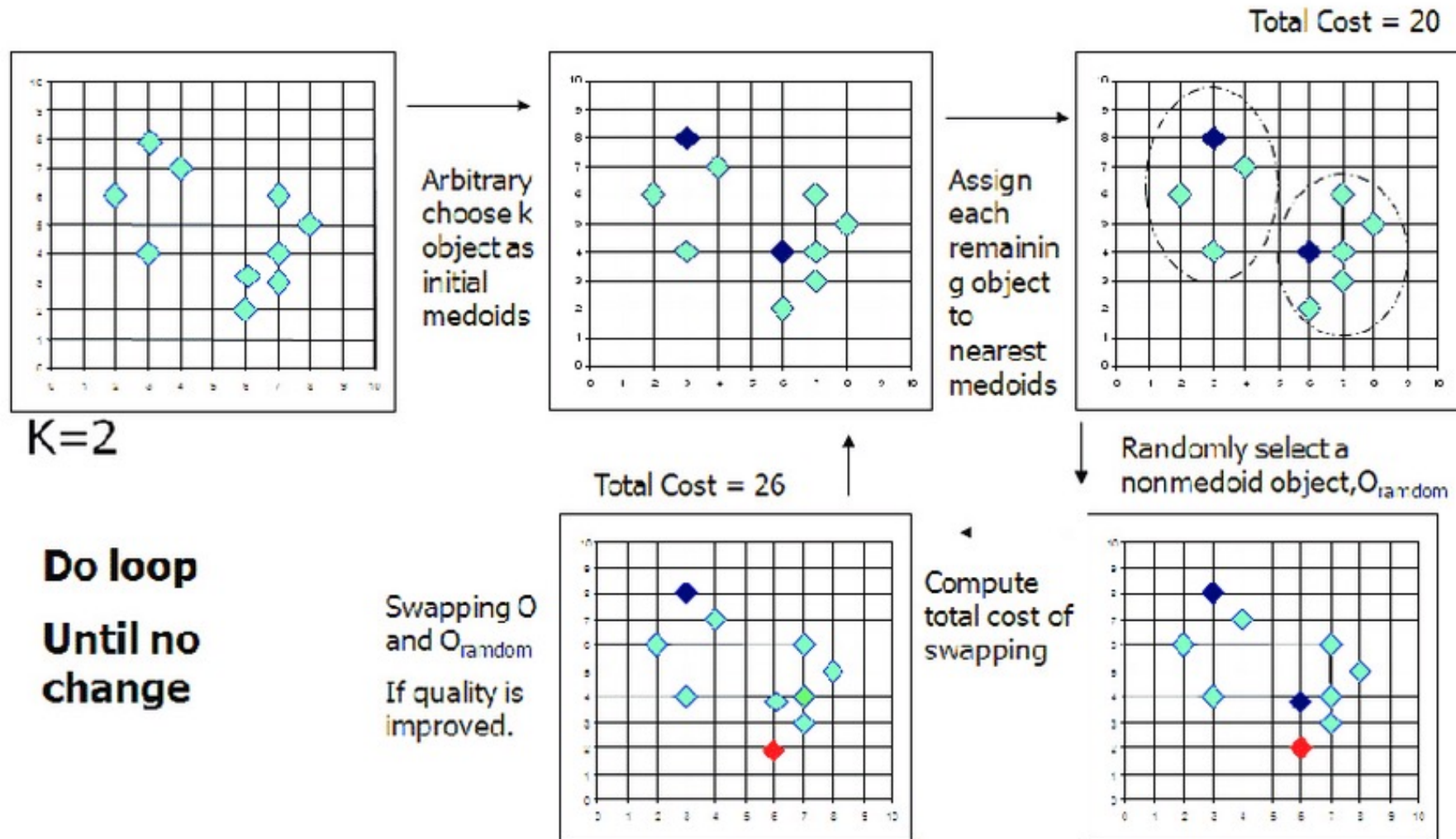
Practical Considerations

- K-Means is preferred for large, well-behaved numerical datasets where speed is crucial and outliers are minimal.
- K-Medoids is chosen when robustness to outliers, flexibility in distance metrics, and interpretability are important, especially for smaller or more complex datasets.

K-Medoids Clustering (PAM)

- Concept: K-Medoids (Partitioning Around Medoids) also divides data into k clusters, but uses actual data points (medoids) as cluster centres. It minimizes the sum of dissimilarities (often **Manhattan distance**) between points and their medoid.
- Algorithm Steps:
 1. Select k initial medoids randomly.
 2. Assign each data point to the nearest medoid.
 3. For each cluster, swap the medoid with another point if it reduces the total distance.
 4. Repeat until no further improvement.
- Strengths: More robust to outliers and noise, flexible with distance metrics, medoids are interpretable as real data points.
- Limitations: Computationally intensive for large datasets, slower than K-Means.

K-Medoids Clustering (PAM)



Hierarchical Clustering

- Hierarchical clustering is a method of unsupervised learning that builds a hierarchy of clusters, organizing data into nested groups.
- Unlike partitioning methods, hierarchical clustering does not require specifying the number of clusters in advance.
- Instead, it produces a tree-like structure called a dendrogram, which visually represents how clusters are merged or split at each step.
- Types of Hierarchical Clustering
 - Agglomerative (Bottom-Up): Each data point starts as its own cluster. At each iteration, the two closest clusters are merged. This process continues until all points are grouped into a single cluster.
 - Divisive (Top-Down): All data points start in one cluster. At each step, the cluster is split into smaller clusters until each point is in its own cluster.

Hierarchical Clustering

- Hierarchical clustering works by iteratively merging the two closest clusters (agglomerative) or splitting the largest cluster (divisive).
- The **linkage criterion** is the rule used to define the "distance" between any two clusters.
- Hierarchical clustering uses different methods to measure the distance between clusters:
 - Single Linkage: Distance between the closest pair of points in two clusters.
 - Complete Linkage: Distance between the farthest pair of points in two clusters.
 - Average Linkage: Average distance between all pairs of points in two clusters.
 - Centroid Linkage: Distance between the centroids of two clusters.

Linkage Criteria

Linkage criteria define how distances between clusters are measured during hierarchical clustering, shaping the resulting cluster structure:

Let C_i and C_j be two clusters with n_i and n_j points, respectively.

Points p_i and p_j belong to clusters C_i and C_j .

The mean of cluster C_i is denoted as m_i . (see next slide)

Linkage criteria define how the distance between clusters is measured:

- **Min (Single Linkage):**

$$D_{\min}(C_i, C_j) = \min_{p_i \in C_i, p_j \in C_j} D(p_i, p_j)$$

(Distance between the closest pair of points.)

Linkage Criteria

- **Max (Complete Linkage):**

$$D_{\max}(C_i, C_j) = \max_{p_i \in C_i, p_j \in C_j} D(p_i, p_j)$$

(Distance between the farthest pair of points.)

- **Mean (Centroid Linkage):**

$$D_{\text{mean}}(C_i, C_j) = \|m_i - m_j\|$$

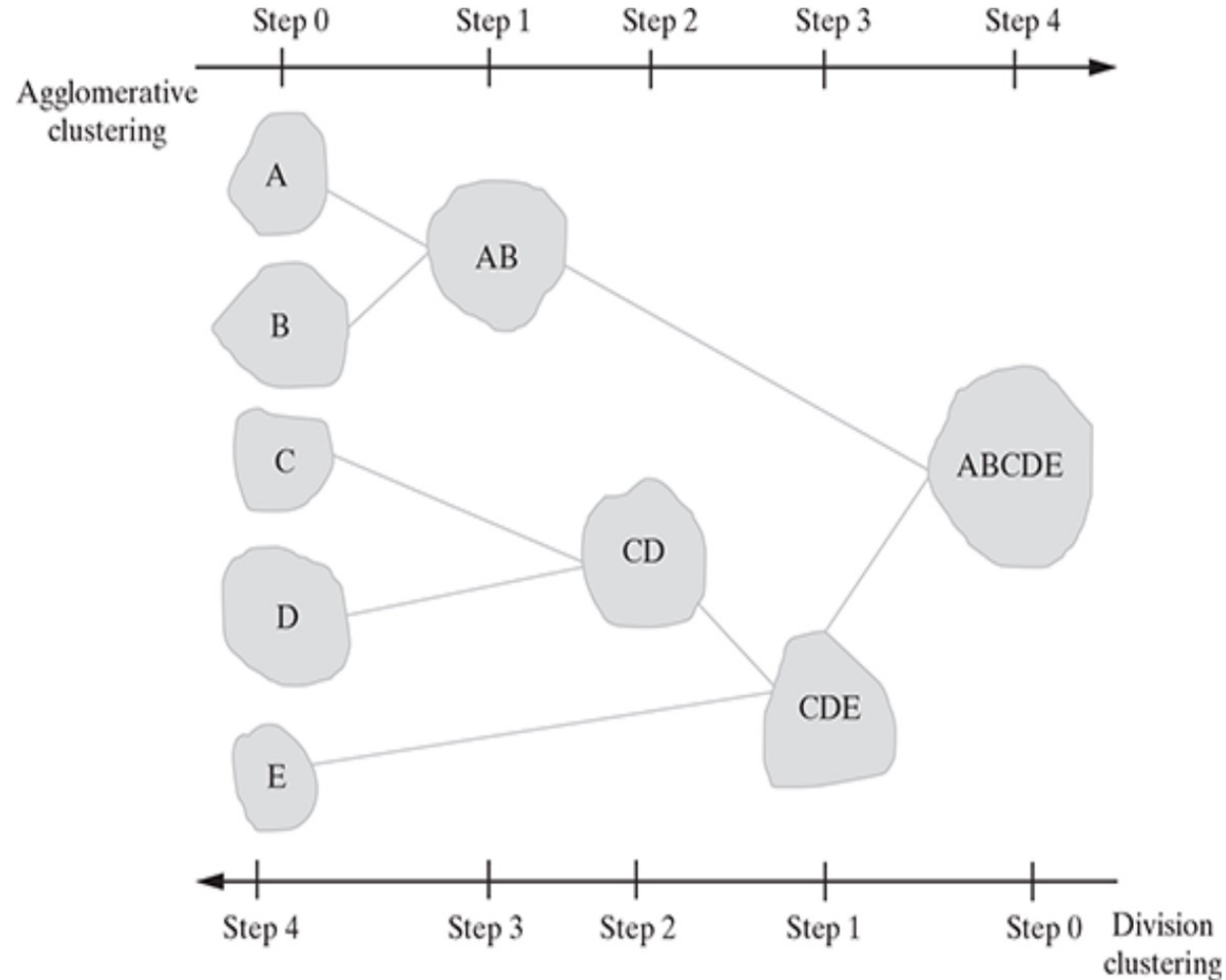
(Distance between cluster means.)

- **Average (Average Linkage):**

$$D_{\text{avg}}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{p_i \in C_i} \sum_{p_j \in C_j} D(p_i, p_j)$$

(Average distance between all pairs of points.)

Agglomerative and divisive hierarchical clustering



Hierarchical Clustering: Agglomerative Algorithm (Bottom-Up)

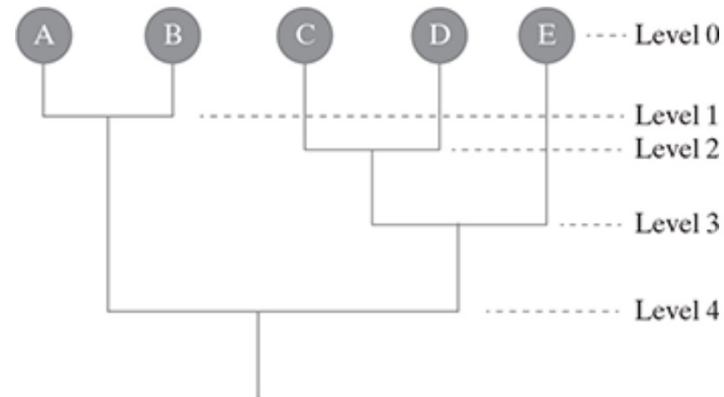
- **Start:** Treat every data point as a separate cluster. (If you have N points, you have N clusters).
- **Find Closest:** Calculate the distance between all pairs of existing clusters using the chosen **Linkage Criterion**. Find the pair with the minimum distance.
- **Merge:** Combine the two closest clusters into a single new cluster.
- **Repeat:** Go back to Step 2 and repeat until all points belong to one big cluster.
- **Result:** A **Dendrogram** (a tree diagram) is built, showing the sequence of merges and the distance at which they occurred.

Hierarchical Clustering: Divisive Algorithm (Top-Down)

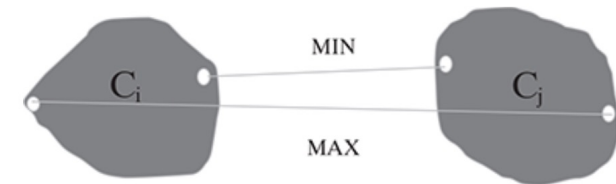
- **Start:** Treat the entire dataset as a single, initial cluster. (1 cluster with N points).
- **Identify Cluster to Split:** Select the current cluster that is the most **heterogeneous** (least similar internally) or the largest.
- **Find the Initial Outlier:** Identify the point in the chosen cluster that is the most dissimilar (farthest) from the other points in that same cluster. This point becomes the start of a new sub-cluster.
- **Recruit Members:** Assign the remaining points to the sub-cluster whose center they are closest to, until no further profitable splits can be made. This results in two new sub-clusters.
- **Repeat:** Go back to Step 2 and repeat the splitting process on the resulting sub-clusters until every data point forms its own cluster (or a pre-defined stopping criterion is met).
- **Result:** A **Dendrogram** is generated, illustrating the sequence of splits.

Dendrograms

- A dendrogram is a binary tree that shows the sequence of merges or splits.
- By cutting the dendrogram at a certain level, you can select the desired number of clusters.
- Dendrograms also help in identifying outliers, as isolated branches often indicate data points that are very different from others.



Dendrogram representation of hierarchical clustering



Distance measure in algorithmic methods

Density based clustering

- **The Limitations of Traditional Clustering**
- You must have observed that methods like K-Means and standard Hierarchical Clustering inherently produce clusters that are **spherical** or nearly convex.
- This is because they rely on measuring the distance to a center (K-Means) or the distance between groups (Hierarchical) across the shortest path, a strategy that fails spectacularly when dealing with real-world, complex data structures.
- If your data forms crescent shapes, interwoven rings, or winding S-shapes, distance-based methods will incorrectly split a single, continuous cluster or merge two distinct ones.
- This limitation motivates **Density-Based Clustering**, which abandons the notion of a center and instead focuses on areas of continuous high data concentration, enabling the discovery of clusters with **arbitrary geometric shapes**.

DBSCAN: Core Concepts and Point Roles

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** is the most popular density-based algorithm.
- It defines clusters based on two mandatory parameters that govern local density:
- **Epsilon** : The maximum radius that defines the neighbourhood of a point.
- **Minimum Points** The minimum number of points required within the epsilon-radius to form a dense region.
- Every data point is classified into one of three critical roles based on these parameters:
- **Core Point**: A point with at least MinPts neighbours within its epsilon-radius. These form the *interior* of a cluster.
- **Border Point**: A point that is within the epsilon-radius of a Core Point, but does not meet the MinPts requirement itself.
- **Noise Point (Outlier)**: A point that is neither a Core Point nor a Border Point.

Cluster Formation and Key Advantages

- A DBSCAN cluster is formed by chaining together **Core Points** that are "density-reachable" from one another.
- If one Core Point's neighbourhood overlaps with another Core Point's neighbourhood, the two are connected, and the cluster grows.
- This process allows the algorithm to trace the *path* of high density through the feature space, enabling it to successfully identify clusters of complex, winding, and nonlinear shapes.
- The main advantages of using DBSCAN are:
 - **Arbitrary Shape Discovery:** It effectively finds non-spherical clusters.
 - **Noise Handling:** It automatically tags data points in sparse regions as **Noise Points** (outliers), making it robust to noisy datasets.
 - **No Predefined K:** It does not require you to pre-specify the number of clusters, as the clusters are discovered organically by the density structure of the data.

Finding Patterns Using Association Rules

- Association rule mining is a powerful unsupervised learning technique for discovering interesting relationships, patterns, and co-occurrences among items in large datasets.
- Association rule presents a methodology that is useful for identifying interesting relationships hidden in large data sets.
- It is also known as association analysis, and the discovered relationships can be represented in the form of association rules comprising a set of frequent items
- The process is widely used in market basket analysis, medical diagnosis, and recommendation systems.

Common terms

- An itemset is a collection of one or more items (e.g., {Milk, Bread, Diaper}).
- A transaction is a record of items purchased together (e.g., a shopping basket).
- k-itemset: An itemset containing k items.

Transaction Number	Purchased Items
1	{Bread, Milk, Egg, Butter, Salt, Apple}
2	{Bread, Milk, Egg, Apple}
3	{Bread, Milk, Butter, Apple}
4	{Milk, Egg, Butter, Apple}
5	{Bread, Egg, Salt}
6	{Bread, Milk, Egg, Apple}

Common terms

2. Support and Frequent Itemsets

- **Support count** (σ): Number of transactions containing an itemset.
- **Support** (s): Fraction of transactions containing an itemset.

$$s(X) = \frac{\sigma(X)}{|T|}$$

- **Frequent itemset**: Itemset with support above a user-defined threshold (*minsup*).

Common terms

3. Association Rules

- An **association rule** is an implication of the form $X \rightarrow Y$, where X and Y are itemsets.
- The rule states that when X occurs, Y is likely to occur as well.

$$\{\text{Milk, Diaper}\} \rightarrow \{\text{Beer}\}$$

Common terms

4. Rule Evaluation Metrics

Support and confidence are the two concepts that are used for measuring the strength of an association rule. Support denotes how often a rule is applicable to a given data set. Confidence indicates how often the items in Y appear in transactions that contain X in a total transaction of N . Confidence denotes the predictive power or accuracy of the rule.

- **Support** (s): Fraction of transactions containing both X and Y .
- **Confidence** (c): Probability that a transaction containing X also contains Y .

$$c(X \rightarrow Y) = \frac{\sigma(X \cup Y)}{\sigma(X)}$$

$$\begin{aligned}\text{Confidence } c(\{\text{Bread, Milk}\} \rightarrow \{\text{Egg}\}) &= \frac{\text{support count of } \{\text{Bread, Milk, Egg}\}}{\text{support count of } \{\text{Bread, Milk}\}} \\ &= \frac{3}{4} \\ &= 0.75\end{aligned}$$

$$\begin{aligned}\text{Confidence } c(\{\text{Bread, Milk}\} \rightarrow \{\text{Egg}\}) &= \frac{\text{support count of } \{\text{Bread, Milk, Egg}\}}{\text{support count of } \{\text{Bread, Milk}\}} \\ &= \frac{3}{4} \\ &= 0.75\end{aligned}$$

Apriori Algorithm: Finding Patterns with Association Rules

- The Apriori algorithm is a foundational method for discovering frequent itemsets and generating association rules in large transactional datasets.
- It is widely used in market basket analysis, retail, and other domains to uncover patterns such as items often bought together.
- Key Steps of the Apriori Algorithm
 1. Set Minimum Support and Confidence Thresholds
 - *Support* measures how frequently an itemset appears in the dataset. *Confidence* measures the likelihood that item Y is purchased when item X is purchased.
 - Example: If the minimum support is 50%, an itemset must appear in at least half of all transactions to be considered frequent.

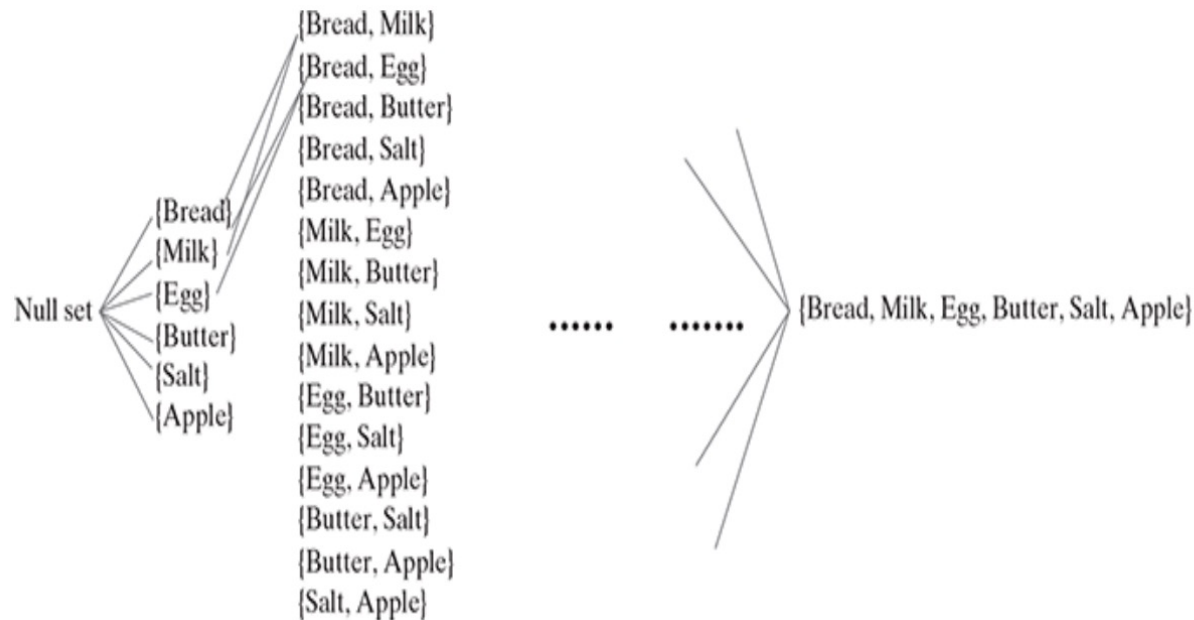
Apriori Algorithm: Finding Patterns with Association Rules

2. Find Frequent 1-Itemsets (L_1)

- Scan the dataset to count occurrences of each item (candidate 1-itemsets, C_1).
- Retain only those meeting the minimum support threshold (frequent 1-itemsets, L_1).

3. Generate Candidate k -Itemsets (C_k) and Prune

- Combine frequent $(k-1)$ -itemsets to form candidate k -itemsets.
- Count their support and prune those below the threshold.
- The algorithm uses the "Apriori property": if a candidate itemset is infrequent, all its supersets will also be infrequent, so they are pruned early.



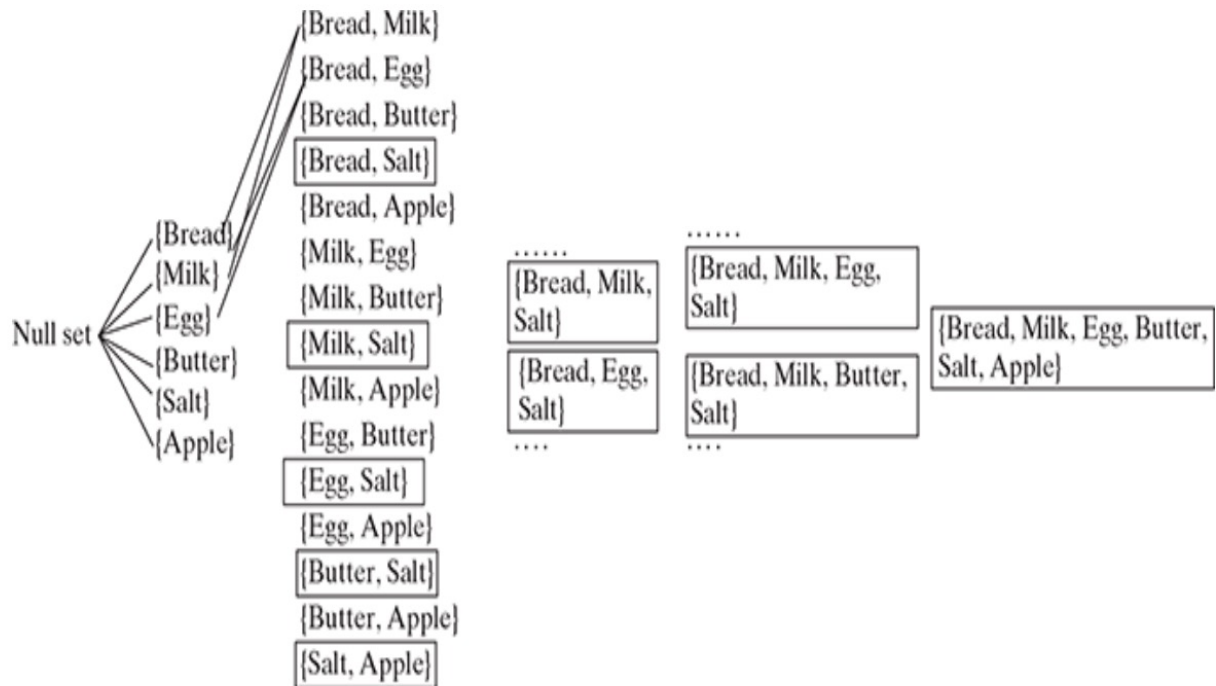
In figure we represents all possible combinations of the 6 items, ranging from the empty set, single-item sets, two-item sets, up to the full set containing all 6 items. Each combination is a candidate itemset that could potentially be frequent, depending on the support threshold.

•Apriori Algorithm Context:

In the context of the Apriori algorithm, not all 64 itemsets need to be checked for frequency. The algorithm uses the Apriori property: if an itemset is infrequent, all its supersets are also infrequent and can be pruned. This dramatically reduces the number of itemsets that must be considered, making the process computationally feasible.

•Join and Prune Steps:

The figure helps illustrate why the join and prune steps are essential. Without pruning, the number of candidate itemsets grows exponentially with the number of items, quickly becoming unmanageable for large datasets.



•Initial Itemset Generation:

All possible itemsets are generated from the available items, including those containing "Salt" (e.g., {Salt}, {Salt, Bread}, {Salt, Milk}, {Salt, Bread, Milk}, etc.).

•Support Threshold Check:

The algorithm checks the support count for each itemset. If "Salt" does not meet the minimum support threshold (i.e., it appears in too few transactions), it is considered infrequent.

•Apriori Property Application:

According to the Apriori property, if an itemset is infrequent, all its supersets (itemsets containing that item) must also be infrequent. Therefore, not only is {Salt} discarded, but all larger itemsets containing "Salt" are also pruned from further consideration.

•Resulting Pruned Search Space:

The figure visually shows the remaining itemsets after all those containing "Salt" have been removed. This dramatically reduces the number of candidate itemsets, making the algorithm more efficient.

Apriori Algorithm: Finding Patterns with Association Rules

4. Iterate Until No More Frequent Itemsets

- Repeat candidate generation and pruning for increasing k until no new frequent itemsets are found.
- This iterative process efficiently narrows down the search space.

5. Generate Association Rules

- For each frequent itemset, generate rules of the form $X \rightarrow Y$.
- Calculate confidence for each rule:

– **Confidence:**

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

- Retain only strong rules that meet the minimum confidence criteria.

Strengths

- Provides reasonable accuracy while working with very large amounts of transactional data
- Discovers rules that are easy to understand
- Provides valuable insight into the unexpected knowledge in data sets, which is a key aspect of learning

Weaknesses

- Not very accurate in the case the data set is small as the smaller occurrences of itemsets may not be due to chance
 - Some effort is involved to separate the insight from the common sense
 - In the case of widespread presence of random patterns, the principle can draw spurious conclusions
-

Example - 1

Let's see an example of the Apriori Algorithm.

Transaction ID	Items
T1	Hot Dogs, Buns, Ketchup
T2	Hot Dogs, Buns
T3	Hot Dogs, Coke, Chips
T4	Chips, Coke
T5	Chips, Ketchup
T6	Hot Dogs, Coke, Chips

Find the **frequent itemsets** and generate **association rules** on this. Assume that minimum support threshold ($s = 33.33\%$) and minimum confident threshold ($c = 60\%$)

$$\begin{aligned}\text{minimum support count} &= \frac{33.33}{100} \times 6 \\ &= 2\end{aligned}$$

Remember

How Support Count is Calculated

The **support count** for an itemset in the Apriori algorithm is calculated by **counting the number of transactions in the dataset that contain that specific itemset**.

1. General Calculation Method

The **Support Count** (or absolute support) for an itemset (X) is:

$$\text{Support Count}(X) = \text{Number of transactions containing all items in } X$$



Item set	Sup-count
Hot Dogs	4
Buns	2
Ketchup	2
Coke	3
Chips	4



Item set	Sup-count
Hot Dogs	4
Buns	2
Ketchup	2
Coke	3
Chips	4



Item set	Sup-count
Hot Dogs, Buns	2
Hot Dogs, Coke	2
Hot Dogs, Chips	2
Coke, Chips	3



Item set	Sup-count
Hot Dogs, Buns	2
Hot Dogs, Ketchup	1
Hot Dogs, Coke	2
Hot Dogs, Chips	2
Buns, Ketchup	1
Buns, Coke	0
Buns, Chips	0
Ketchup, Coke	0
Ketchup, Chips	1
Coke, Chips	3



Item set	Sup-count
Hot Dogs, Buns, Coke	0
Hot Dogs, Buns, Chips	0
Hot Dogs, Coke, Chips	2



Item set	Sup-count
Hot Dogs, Coke, Chips	2

There is only one itemset with minimum support 2. So only one itemset is frequent.

Frequent Itemset (I) = {Hot Dogs, Coke, Chips}

Association Rules

- $[Hot\ Dogs \wedge Coke] \implies [Chips]$
Confidence = $\frac{sup(Hot\ Dogs \wedge Coke \wedge Chips)}{sup(Hot\ Dogs \wedge Coke)} = \frac{2}{2} \times 100\% = 100\%$ (*Selected*)
- $[Hot\ Dogs \wedge Chips] \implies [Coke]$
Confidence = $\frac{sup(Hot\ Dogs \wedge Coke \wedge Chips)}{sup(Hot\ Dogs \wedge Chips)} = \frac{2}{2} \times 100\% = 100\%$ (*Selected*)
- $[Coke \wedge Chips] \implies [Hot\ Dogs]$
Confidence = $\frac{sup(Hot\ Dogs \wedge Coke \wedge Chips)}{sup(Coke \wedge Chips)} = \frac{2}{3} \times 100\% \approx 66.67\%$ (*Selected*)
- $[Hot\ Dogs] \implies [Coke \wedge Chips]$
Confidence = $\frac{sup(Hot\ Dogs \wedge Coke \wedge Chips)}{sup(Hot\ Dogs)} = \frac{2}{4} \times 100\% = 50\%$ (*Rejected*)
- $[Coke] \implies [Hot\ Dogs \wedge Chips]$
Confidence = $\frac{sup(Hot\ Dogs \wedge Coke \wedge Chips)}{sup(Coke)} = \frac{2}{3} \times 100\% \approx 66.67\%$ (*Selected*)
- $[Chips] \implies [Hot\ Dogs \wedge Coke]$
Confidence = $\frac{sup(Hot\ Dogs \wedge Coke \wedge Chips)}{sup(Chips)} = \frac{2}{4} \times 100\% = 50\%$ (*Rejected*)

There are four strong results (minimum confidence greater than 60%).

1	Tid	Items Bought	2	Items Bought	Support
	1	Milk, Tea, Cake		Milk	2
	2	Eggs, Tea, Cold Drink		Eggs	3
	3	Milk, Eggs, Tea, Cold Drink		Tea	3
	4	Eggs, Cold Drink		Cold Drink	3
	5	Juice		Juice	1
				Cake	1

5	Items Bought	Support	4	Items Bought	3	Items Bought	Support
	Milk, Eggs	1		Milk, Eggs		Milk	2
	Milk, Tea	2		Milk, Tea		Eggs	3
	Milk, Cold Drink	1		Milk, Cold Drink		Tea	3
	Eggs, Tea	2		Eggs, Tea		Cold Drink	3
	Eggs, Cold Drink	3		Eggs, Cold Drink			
	Tea, Cold Drink	2		Tea, Cold Drink			

6	Items Bought	Support	7	Items Bought	Support
	Milk, Tea	2		Eggs, Tea, Cold Drink	2
	Eggs, Tea	2			
	Eggs, Cold Drink	3			
	Tea, Cold Drink	2			

There is only one itemset with minimum support 2.
So only one itemset is frequent